

Singly Linked List and Doubly Linked List

1. Singly Linked List

Definition:

A **Singly Linked List** is a linear data structure where each node contains:

1. **Data:** The value stored in the node.
2. **Next Pointer:** A pointer to the next node in the list.

Example:

Let's consider the following **Singly Linked List**:

Copy

Head -> 10 -> 20 -> 30 -> NULL

1. Insertion in a Singly Linked List

Case 1: Insertion at the Beginning

Example: Insert **5** at the beginning.

Step-by-Step Explanation:

1. Create a new node with data **5**.
2. Set the **next pointer** of the new node to point to the current head (node with data **10**).

Copy

New Node: 5 -> 10

3. Update the **head** of the list to point to the new node.

Copy

Head -> 5 -> 10 -> 20 -> 30 -> NULL

Case 2: Insertion at the End

Example: Insert **40** at the end.

Step-by-Step Explanation:

1. Create a new node with data **40**.
2. Traverse the list to reach the last node (node with data **30**).
3. Set the **next pointer** of the last node to point to the new node.

Copy

30 -> 40

4. Set the **next pointer** of the new node to NULL.

Copy

Head -> 5 -> 10 -> 20 -> 30 -> 40 -> NULL

Case 3: Insertion at a Specific Position

Example: Insert **15** at position **3** (after node with data **10**).

Step-by-Step Explanation:

1. Create a new node with data **15**.
2. Traverse the list to reach the node at position **2** (node with data **10**).
3. Set the **next pointer** of the new node to point to the next node of the current node (node with data **20**).

Copy

15 -> 20

4. Set the **next pointer** of the current node to point to the new node.

Copy

10 -> 15 -> 20

5. Updated list:

Copy

Head -> 5 -> 10 -> 15 -> 20 -> 30 -> 40 -> NULL

2. Deletion in a Singly Linked List

Case 1: Deletion at the Beginning

Example: Delete the first node.

Step-by-Step Explanation:

1. Check if the list is empty. If empty, return.
2. Store the head node (node with data **5**) in a temporary variable.
3. Update the **head** to point to the next node (node with data **10**).

Copy

Head -> 10 -> 15 -> 20 -> 30 -> 40 -> NULL

4. Free the memory of the temporary node.
-

Case 2: Deletion at the End

Example: Delete the last node.

Step-by-Step Explanation:

1. Check if the list is empty. If empty, return.
2. Traverse the list to reach the second last node (node with data **30**).
3. Set the **next pointer** of the second last node to NULL.

Copy

30 -> NULL

4. Free the memory of the last node.

Copy

Head -> 10 -> 15 -> 20 -> 30 -> NULL

Case 3: Deletion at a Specific Position

Example: Delete the node at position **3** (node with data **20**).

Step-by-Step Explanation:

1. Check if the list is empty. If empty, return.
2. Traverse the list to reach the node just before the desired position (node with data **15**).
3. Store the node to be deleted (node with data **20**) in a temporary variable.
4. Set the **next pointer** of the current node to point to the next node of the node to be deleted (node with data **30**).

Copy

15 -> 30

5. Free the memory of the temporary node.

Copy

Head -> 10 -> 15 -> 30 -> NULL

3. Display Operation in a Singly Linked List

Example: Display the list.

Step-by-Step Explanation:

1. Start from the head of the list (node with data **10**).
2. Traverse the list by following the **next pointers**.
3. Print the data of each node during traversal:

Copy

10 -> 15 -> 30 -> NULL

2. Doubly Linked List

Definition:

A **Doubly Linked List** is a linear data structure where each node contains:

1. **Data:** The value stored in the node.
2. **Next Pointer:** A pointer to the next node in the list.
3. **Previous Pointer:** A pointer to the previous node in the list.

Example:

Let's consider the following **Doubly Linked List**:

Copy

Head <-> 10 <-> 20 <-> 30 <-> NULL

1. Insertion in a Doubly Linked List

Case 1: Insertion at the Beginning

Example: Insert **5** at the beginning.

Step-by-Step Explanation:

1. Create a new node with data **5**.
2. Set the **next pointer** of the new node to point to the current head (node with data **10**).

Copy

5 -> 10

3. Set the **previous pointer** of the new node to NULL.
4. Set the **previous pointer** of the current head to point to the new node.

Copy

5 <-> 10

5. Update the **head** of the list to point to the new node.

Copy

Head <-> 5 <-> 10 <-> 20 <-> 30 <-> NULL

Case 2: Insertion at the End

Example: Insert **40** at the end.

Step-by-Step Explanation:

1. Create a new node with data **40**.
2. Traverse the list to reach the last node (node with data **30**).
3. Set the **next pointer** of the last node to point to the new node.

Copy

30 -> 40

4. Set the **previous pointer** of the new node to point to the last node.

Copy

30 <-> 40

5. Set the **next pointer** of the new node to NULL.

Copy

Head <-> 5 <-> 10 <-> 20 <-> 30 <-> 40 <-> NULL

Case 3: Insertion at a Specific Position

Example: Insert **15** at position **3** (after node with data **10**).

Step-by-Step Explanation:

1. Create a new node with data **15**.
2. Traverse the list to reach the node at position **2** (node with data **10**).
3. Set the **next pointer** of the new node to point to the next node of the current node (node with data **20**).

Copy

15 -> 20

4. Set the **previous pointer** of the new node to point to the current node.

Copy

10 <-> 15

5. Set the **previous pointer** of the next node to point to the new node.

Copy

15 <-> 20

6. Set the **next pointer** of the current node to point to the new node.

Copy

10 <-> 15 <-> 20

7. Updated list:

Copy

Head <-> 5 <-> 10 <-> 15 <-> 20 <-> 30 <-> 40 <-> NULL

2. Deletion in a Doubly Linked List

Case 1: Deletion at the Beginning

Example: Delete the first node.

Step-by-Step Explanation:

1. Check if the list is empty. If empty, return.
2. Store the head node (node with data **5**) in a temporary variable.
3. Update the **head** to point to the next node (node with data **10**).

Copy

Head <-> 10 <-> 15 <-> 20 <-> 30 <-> 40 <-> NULL

4. Set the **previous pointer** of the new head to NULL.
5. Free the memory of the temporary node.

Case 2: Deletion at the End

Example: Delete the last node.

Step-by-Step Explanation:

1. Check if the list is empty. If empty, return.
2. Traverse the list to reach the second last node (node with data **30**).
3. Set the **next pointer** of the second last node to NULL.

Copy

30 -> NULL

4. Free the memory of the last node.

Copy

Head <-> 10 <-> 15 <-> 20 <-> 30 <-> NULL

Case 3: Deletion at a Specific Position

Example: Delete the node at position **3** (node with data **20**).

Step-by-Step Explanation:

1. Check if the list is empty. If empty, return.
2. Traverse the list to reach the node to be deleted (node with data **20**).
3. Set the **next pointer** of the previous node to point to the next node of the node to be deleted.

Copy

15 -> 30

4. Set the **previous pointer** of the next node to point to the previous node of the node to be deleted.

Copy

15 <-> 30

5. Free the memory of the node to be deleted.

Copy

Head <-> 10 <-> 15 <-> 30 <-> NULL

3. Display Operation in a Doubly Linked List

Example: Display the list.

Step-by-Step Explanation:

1. Start from the head of the list (node with data **10**).
2. Traverse the list by following the **next pointers**.
3. Print the data of each node during traversal:

Copy

10 <-> 15 <-> 30 <-> NULL

Summary of Operations

Singly Linked List:

1. **Insertion:**

- At the beginning
- At the end
- At a specific position

2. **Deletion:**

- At the beginning
- At the end
- At a specific position

3. **Display:** Traverse and print the list.

Doubly Linked List:

1. **Insertion:**

- At the beginning
- At the end
- At a specific position

2. **Deletion:**

- At the beginning
- At the end
- At a specific position

3. **Display:** Traverse and print the list.